

数组

中山大学计算机学院



主讲人：潘茂林



目录

CONTENTS

01

数组的概念

02

数组的定义与初始化

03

数组相关操作

04

多维数组

05

数组与函数



数组 (Array) 的概念

向量与序列数据如何表示:

- **向量(vector)**, 例如: 2d向量, 3d向量:

$(x, y); (x, y, z)$

- **序列(sequence)**, 高维向量, 例如:

$(a_1, a_2, a_3, \dots, a_i, \dots a_j, \dots a_n)$

- **文字(literal)**, 例如:

" Doe, a deer, a female deer\n Ray, a drop of golden sun "

这里, a_i 是向量或序列的一个**元素(element)**或**项(item)**; i 是下标(*index*), 表示第几个元素; n 是**长度**, 表示向量或序列由多少个元素组成。



数组 (Array) 的概念 - 应用案例

某市气象局记录了一个月降水数据，例如：

0,3.5,0,0,0,0,0,0,0,0,4.2,0,0,0,0,0,0,0,0,7.0,0,0,0,0,0,0,0,-1

请计算该市一次降水的平均值？

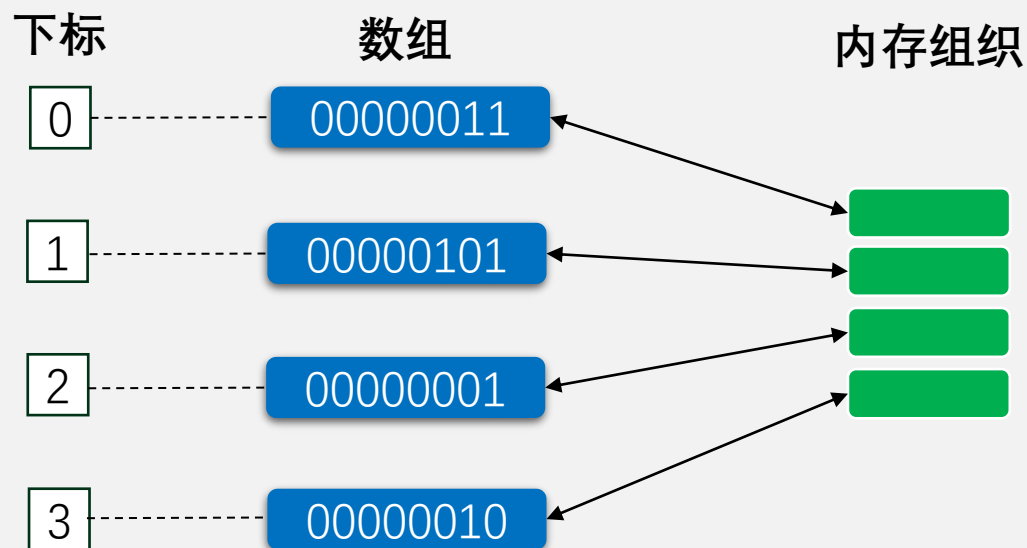
C语言



数组 (Array) 的概念 - 一种数据结构

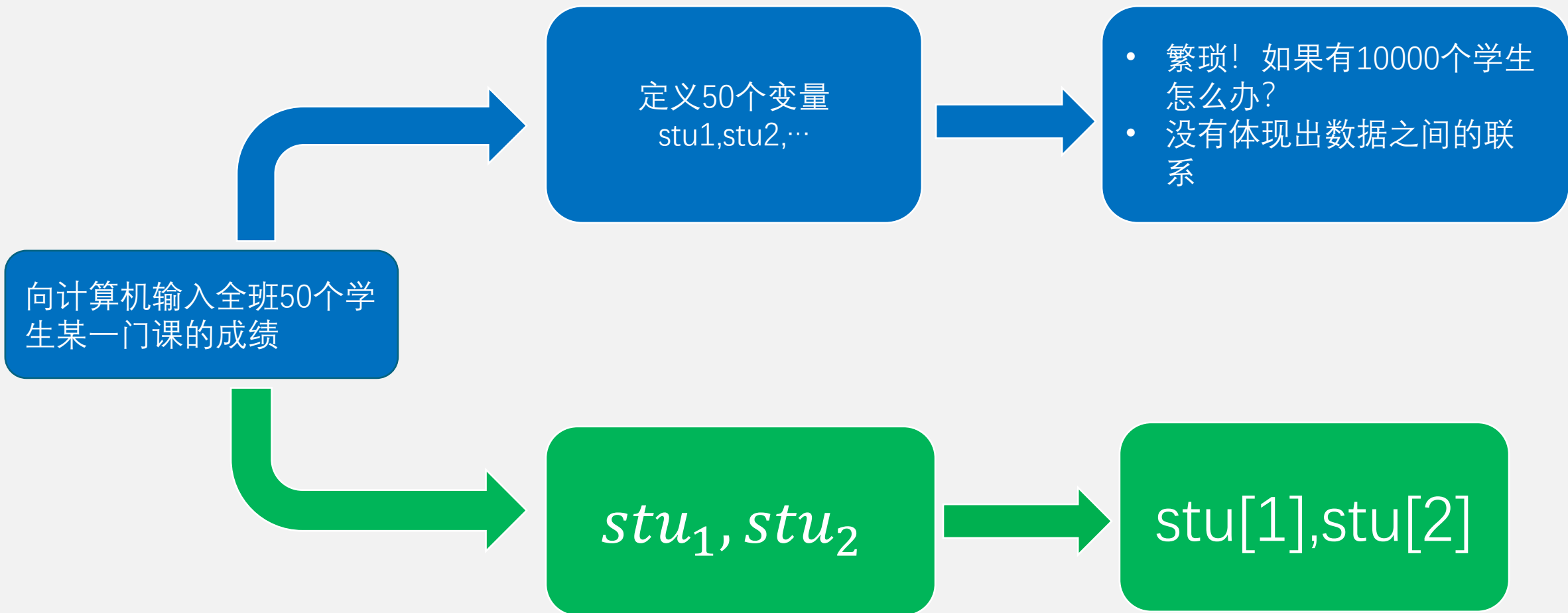
在计算机科学中，**数组数据结构 (array data structure)**，简称**数组 (Array)**，是由**相同类型**的元素 (**element**) 的集合所组成的数据结构，分配一块**连续**的内存来存储。利用元素的**索引/下标 (index)** 可以计算出该元素对应的存储地址。

最简单的数据结构类型是**一维数组**。一个**数组类型**描述了连续分配的非空的具有特定元素对象类型的对象集合。这些元素对象的类型称为元素类型 (element type)。**数组类型由元素类型与元素的数目确定。**





数组 (Array) 的概念 - 为什么要使用数组





一维数组的定义

类型说明符 数组名[整数常量表达式]

int stu[50]

stu[0]	stu[1]	stu[2]	stu[3]	stu[4]	……	stu[46]	stu[47]	stu[48]	stu[49]
--------	--------	--------	--------	--------	----	---------	---------	---------	---------

- 注意！**C语言数组的下标从0开始！** 不存在数组元素stu[50]
- 在定义数组时，需要指定数组中元素的个数，方括号中的常量表达式用来表示元素的个数，即**数组长度**



一维数组的初始化

1. 对所有元素赋值，例如 `int a[5]={0,1,2,3,4}`
2. 对部分元素赋值，例如 `int a[5]={0,1}`，此时**剩余的元素会被自动赋0值**
3. 不指定数组长度，例如 `int a[]={0,1,2}`，它等价于 `int a[3]={0,1,2}`。注意：**不是可变长度数组**，仅是由编译器根据初始值数量自动确定长度。
4. C99 开始，支持可变长度数组。

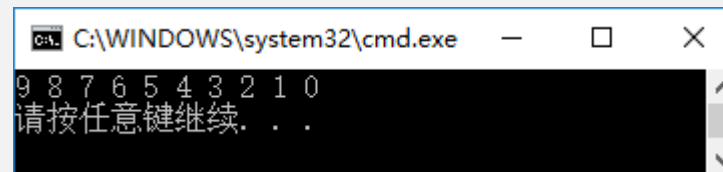


数组操作 - 使用下标访问数组元素

运算符	运算符名	示例	描述
[]	数组下标	a [b]	访问数组 a 的第 b 个元素

对10个数组元素依次赋值为0,1,2,3,4,5,6,7,8,9，要求按逆序输出。

```
#include<stdio.h>
int main()
{
    int i,a[10];
    for(i=0; i<=9;i++) //对数组元素a[0]~a[9]赋值
        a[i]=i;
    for(i=9;i>=0;i--) //输出a[9]~a[0]共10个数组元素
        printf("%d ",a[i]);
    printf("\n");
    return 0;
}
```



第1个for循环使a[0] ~ a[9]的值为0 ~ 9。

第2个for循环按a[9]~a[0]的顺序输出各元素的值。



数组操作 - 应用案例

某市气象局记录了一个月降水数据，例如：

0,3.5,0,0,0,0,0,0,0,0,4.2,0,0,0,0,0,0,0,0,7.0,0,0,0,0,0,0,0,-1

请计算该市一次降水的平均值？

```
/*average rain*/
#include<stdio.h>

int main() {
    double data[100];
    int n=0;

    /*读入一个数组，以 -1 作为结束*/
    do {
        scanf("%lf", &data[n]);
        n++;
    } while (data[n-1] != -1);
    n--;
```

```
int rainedays = 0;
double rain_total = 0;
for (int i=0; i<n; i++) {
    if (data[i] > 1e-7) {
        rainedays++;
        rain_total += data[i];
    }
}

if (rainedays)
    printf("Rainedays %d, average rainfall %lf\n", rainedays, rain_total/rainedays);
else
    printf("No rain.\n");
}
```



数组常见定义错误

```
#define N 5

int main() {
    int arr1[]; //没有声明元素数量

    int m=5;
    int arr2[m]; //C99可变长度数组/VLA（仅适用自动变量）

    int arr3[N];
    arr3 = {1,2,3,4,5}; // 数组标识符是值（地址），不能作为左值

    int arr4[N] = {1,2,3,4,5,6}; //初始化超长度（warning）

    int arr5[N] = {1,2,3,4,5}, a[N];
    a = arr5; // a 不是左值

    return 0;
}
```



数组常见处理

- 需要获得数组的长度

```
#define LEN_ARRAY(a) (sizeof(a) / sizeof(a[0]))
```

- `sizeof(array)` 返回该数组定义的字节 (bytes) 数
- 类型隐式转换
 - `int a[] = {1, 2, 3.14, 4, 5};`
- 数组名是地址值 (右值), 它也是其第一个元素的地址, 即
 - `a == &(a[0])`
 - 因此, `*a == *&(a[0]) == a[0]`, 是左值
- 数组元素赋零值
 - `memset(arr, 0, sizeof(arr));`



二维数组简介

现在有一个3*3的矩阵，需要保存在计算机当中。

如果创建一个数组，应当是二维的，第一维代表第几行，第二维代表第几列。

类型说明符 数组名[常量表达式][常量表达式]

```
int mat[3][3]
```

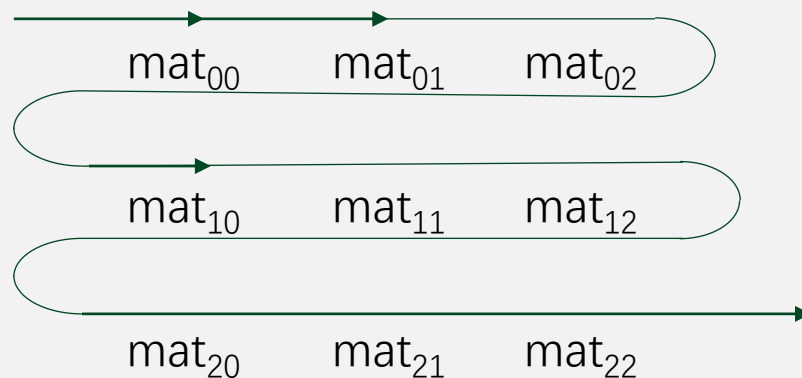
mat[0] —— mat[0][0] mat[0][1] mat[0][2]

mat[1] —— mat[1][0] mat[1][1] mat[1][2]

mat[2] —— mat[2][0] mat[2][1] mat[2][2]

二维数组存储

```
int mat[3][3]
```



- 用**矩阵形式**（如3行3列形式）表示二维数组，是**逻辑**上的概念，能形象地表示出行列关系。而在**内存**中，各元素是连续存放的，不是二维的，是**线性**的。



二维数组的初始化

(1)分行给二维数组赋初值。

```
int mat[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

(2)可以将所有数据写在一个花括号内，按数组元素在内存中的排列顺序对各元素赋初值。

```
int mat[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

对于多维数组，除第一维外其他维的长度均不可省略

(3)可以对部分元素赋初值。

```
int mat[3][4]={{1},{5},{9}};
```

①

```
int mat[3][4]={{1},{0,6},{0,0,11}};
```

②

```
int mat[3][4]={{1},{5,6}};
```

③

```
int mat[3][4]={{1},{},{9}};
```

④

①	②	③	④
1 0 0 0	1 0 0 0	1 0 0 0	1 0 0 0
5 0 0 0	0 6 0 0	5 6 0 0	0 0 0 0
9 0 0 0	0 0 11 0	0 0 0 0	9 0 0 0

(4)如果对全部元素都赋初值(即提供全部初始数据)，则定义数组时对**第1维的长度可以不指定**，但第2维的长度不能省。

```
int mat[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

≡

```
int mat[][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

在定义时也可以只对部分元素赋初值而省略第1维的长度，但应分行赋初值。

```
int mat[][4]={{0,0,3},{},{0,10}};
```



数组与函数

如何将数组作为参数传入某个函数？ 例如：计算两个向量的和

```
/*vector add*/
#include<stdio.h>
#define LEN_ARR(a) (sizeof(a) / sizeof(a[0]))
void vadd(int a[const], const int b[], const size_t sz);

int main(){
    int a[9] = {1,2,3,4,5,6,7,8,9};
    vadd(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++)
        printf("%d,",a[i]);
    printf("\n");
}

void vadd(int a[const], const int b[], const size_t sz){
    for (int i=0;i<sz;i++) {
        a[i] += b[i];
//        a++; //increment of read-only parameter 'a'
//        b[i] = 1; //assignment of read-only location
    }
}
```

要点：

① 数组形参是一个变量，可保存数组的地址，用不定长数组表示，如 `b[]`。如果形参定义为 `int b[10]`，则编译忽略数组长度。

② **const** 在函数申明中应用

- `int a[const]` 表示形参 `a` 不可修改
- `const int b[]` 表示 `b` 的元素不可修改
- `const size_t sz` 表示 `sz` 不可修改

③ 函数中无法获得数组的长度，通常会将数组长度作为参数传给函数，类型是 `size_t`



数组与函数

如何将多维数组作为参数传入某个函数？

```
/*matrix add*/
#include<stdio.h>
... ..
void matrix_add(int a[][3], int b[][3], size_t m);

int main(){
    int a[3][3] = {1,2,3,4,5,6,7,8,9};
    matrix_add(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++) {
        for (int j=0;j<LEN_ARR(a[i]);j++)
            printf("%4d,",a[i][j]);
        printf("\n");
    }
}

void matrix_add(int a[][3], int b[][3], size_t m){
    vadd(a,b,m*LEN_ARR(a[0]));
}
```

要点：

- ① 多维数组形参仅**第一维**用不定长数组表示，如 b[][3]。
- ② 如果形参定义为 int b[10][3], 则编译解释 int b[][3]。
- ③ 多维数组形参除第一维外，长度必须是常量



数组与函数

案例程序中，三种函数声明都能处理 3x3 矩阵：

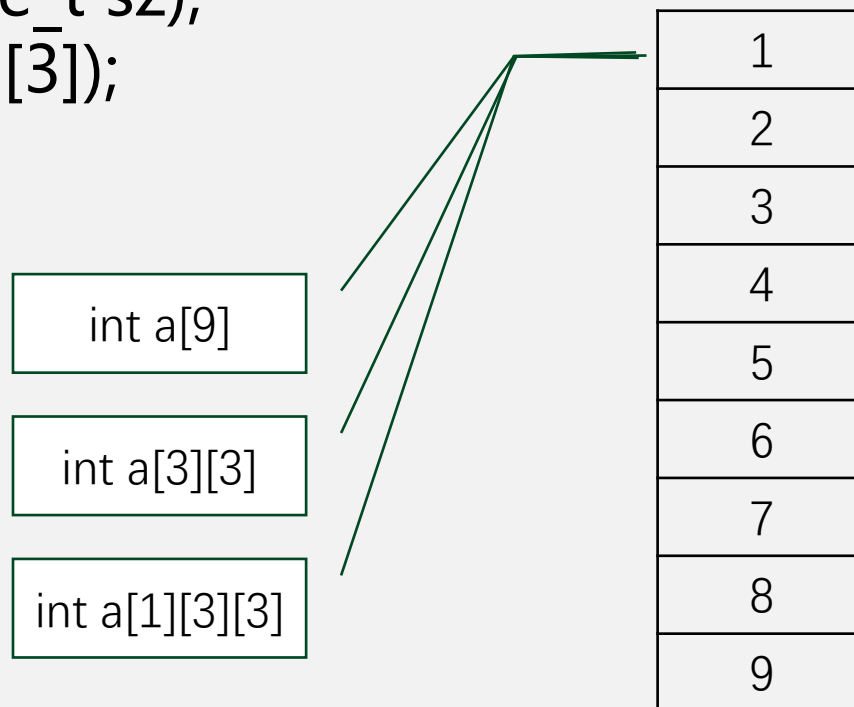
```
void vadd(int a[const], const int b[], const size_t sz);
```

```
void matrix_add(int a[][3], int b[][3], size_t sz);
```

```
void matrix_add1(int a[][3][3], int b[][3][3]);
```

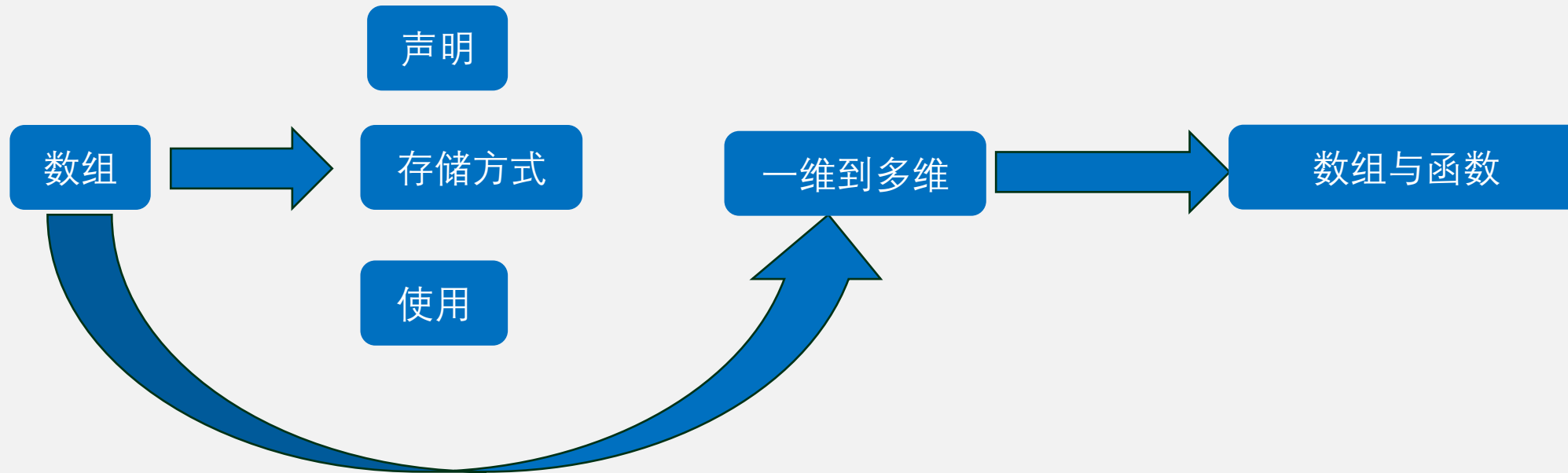
你认为哪种比较最合适呢？

例如：加法，点积，外积





总结





中山大學
SUN YAT-SEN UNIVERSITY



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



编制人：课题组